

Aula 7 (Módulo 01) — Comandos de investigação: grep, awk, sed, find, lsof

Esta é a aula de consolidação do Módulo 01. Os comandos anteriores (`ps`, `ss`, `journalctl` ...) **mostram** informação; os desta aula servem para **filtrar, transformar e localizar** — normalmente combinados com os anteriores usando o **pipe** (`|`), que já usamos várias vezes sem formalizar: ele pega a saída de um comando e entrega como entrada do próximo.

1. `grep` — encontrar linhas que combinam com um padrão

```
grep "Failed password" /var/log/auth.log
```

Procura, dentro do arquivo, todas as linhas que contêm o texto `Failed password` e mostra só elas. Flags úteis:

- `-i` → ignora maiúsculas/minúsculas
- `-v` → **inverte**: mostra as linhas que **não** combinam
- `-c` → mostra só a **contagem** de linhas que combinam
- `-E` → habilita regex estendida (padrões mais poderosos, ex.: `grep -E "Failed|Invalid"` busca por qualquer uma das duas palavras)
- `-r` → busca recursivamente em todos os arquivos de uma pasta

Exemplo de uso real de investigação:

```
grep "Failed password" /var/log/auth.log | grep -c "203.0.113.45"
```

Isso conta quantas vezes o IP `203.0.113.45` falhou login — um jeito rápido de confirmar um possível brute force que vimos na Aula 4 (SSH).

2. `awk` — extrair e processar colunas

`awk` trata cada linha como uma sequência de **colunas** (separadas por espaço, por padrão) e permite extrair/processar só o que interessa.

```
awk '{print $1, $9}' arquivo.log      # imprime só a coluna 1 e a coluna 9
```

Exemplo real: extrair só os IPs de origem de tentativas de login SSH falhadas (a posição exata da coluna do IP varia conforme o formato do log, mas o princípio é este):

```
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn
```

Isso: filtra as falhas (`grep`), extrai o campo do IP (`awk`, usando `NF-3` = "a 4ª coluna a partir do fim", uma forma de pegar um campo mesmo sem contar posições manualmente), ordena (`sort`), conta ocorrências únicas (`uniq -c`) e ordena da maior contagem para a menor (`sort -rn`) — resultado: **quais IPs mais tentaram login, do que mais tentou**

para o que menos tentou. Isso é, literalmente, o tipo de análise que uma query de SIEM faz automaticamente — aqui estamos fazendo "na mão" para entender o princípio por trás.

3. `sed` — substituir/editar texto em fluxo

```
sed 's/senha_antiga/senha_nova/' arquivo.txt # substitui a 1ª ocorrência por linha
sed 's/senha_antiga/senha_nova/g' arquivo.txt # substitui TODAS as ocorrências (g = global)
```

Em investigação, `sed` é menos usado para "ler" logs (isso é mais função do `grep` / `awk`) e mais para **normalizar/limpar** dados antes de analisar — por exemplo, remover timestamps para comparar só o conteúdo de várias linhas, ou anonimizar dados sensíveis antes de compartilhar um trecho de log em um relatório.

4. `find` — localizar arquivos por critério

```
find /home -name "*.sh" # arquivos .sh dentro de /home
find / -mtime -1 -type f 2>/dev/null # arquivos modificados nas últimas 24h
find / -perm -4000 2>/dev/null # arquivos com bit SUID ativo
```

O último exemplo merece atenção: bit **SUID** (`-perm -4000`) faz um executável rodar **com a permissão do dono do arquivo**, não de quem o executa — se o dono for `root`, qualquer usuário que rode aquele programa ganha, temporariamente, privilégios de root **durante a execução**. É uma funcionalidade legítima do Linux (usada por programas como `passwd`, que precisa escrever em `/etc/shadow` mesmo quando executado por um usuário comum), mas também é um dos vetores clássicos de **escalada de privilégio**: um SUID mal configurado (ou propositalmente plantado por um atacante) em um binário que permite executar comandos arbitrários é um caminho direto para virar root.

Por que importa para segurança: `find / -perm -4000` é literalmente um comando de checklist de hardening/hunting real — times de segurança rodam isso periodicamente para garantir que só os binários SUID esperados (do próprio sistema operacional) existam, e investigar qualquer um novo ou inesperado.

5. `lsof` — quais arquivos/conexões um processo tem abertos

`lsof` = *list open files*. No Linux, lembre-se do princípio da Aula 1 deste módulo ("quase tudo é arquivo") — isso inclui conexões de rede.

```
sudo lsof -p 1234 # tudo que o processo de PID 1234 tem aberto
sudo lsof -i :22 # qual processo está usando a porta 22
sudo lsof /var/log/auth.log # quais processos têm esse arquivo aberto agora
```

Por que importa para segurança: `lsof -i` é um complemento direto do `ss -tulpn` que vimos na aula passada — enquanto `ss` foca na conexão, `lsof` foca no processo e em **todos** os recursos que ele tem abertos (arquivos, sockets, pipes), o que ajuda a responder "esse processo suspeito está com qual arquivo/conexão aberta agora, exatamente?".

6. Conectando tudo — um exemplo de investigação combinada

```
# 1. Confirmar padrão de brute force
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head -5

# 2. Ver se algum desses IPs teve sucesso depois
grep "Accepted" /var/log/auth.log | tail -20

# 3. Se algo suspeito rodando, ver o que esse processo tem aberto
sudo lsof -p <PID_suspeito>

# 4. Procurar por SUID inesperado como sinal de escalada de privilégio
find / -perm -4000 -type f 2>/dev/null
```

Isso não é um exercício isolado — é essencialmente um mini-roteiro de **Threat Hunting** (Módulo 14) e de **Incident Response** (Módulo 15), só que usando comandos "crus" de Linux em vez de um SIEM/EDR. Entender esse roteiro na mão é o que torna o uso de ferramentas prontas (SIEM, EDR) muito mais intuitivo depois — você vai reconhecer que elas estão automatizando exatamente este tipo de raciocínio.

7. Exercício prático

```
find /etc -perm -4000 -type f 2>/dev/null
```

(usar `/etc` em vez de `/` deixa o exercício rápido; na prática, uma varredura real seria em `/` inteiro)

```
sudo lsof -i -P -n | head -15
```

(`-P -n` evita que o `lsof` tente resolver nomes de porta/host, deixando mais rápido e mostrando números diretos)

Compare a saída do `lsof -i` com o que você já tinha visto com `ss -tulpn` na aula anterior — são visões complementares da mesma realidade.

8. Recapitulando o Módulo 01

- `grep` filtra linhas por padrão; `awk` extrai/processa colunas; `sed` substitui texto. Combinados, transformam um log bruto em uma resposta específica (ex.: "quais IPs mais tentaram brute force").
- `find` localiza arquivos por critério — `-perm -4000` (SUID) é uma checagem clássica de escalada de privilégio.
- `lsof` mostra o que um processo tem aberto (arquivos e conexões), complementando `ss`.
- Com as 7 aulas deste módulo, você já sabe: navegar o filesystem, ler/ alterar permissões, investigar processos e persistência (systemd/ cron), entender SSH, ler logs (syslog/journalctl) e agora filtrar/ localizar evidências — a base completa para investigar um host Linux.

Módulo 01 concluído. 🎉

Próximo módulo

Módulo 02 — Windows para Blue Team: mesma lógica, mas para o sistema operacional mais comum em ambientes corporativos — arquitetura, processos, Registry, PowerShell, Event Viewer, Sysmon.

Fontes recomendadas

- **GNU Grep, Awk, Sed — manuais oficiais:** referência completa de cada ferramenta.
<https://www.gnu.org/software/grep/manual/>, <https://www.gnu.org/software/gawk/manual/>,
<https://www.gnu.org/software/sed/manual/sed.html>
- **Linux man-pages — `find(1)`, `lsuf(8)`:** referência oficial. <https://man7.org/linux/man-pages/man1/find.1.html>
- **MITRE ATT&CK — T1548.001 (Abuse Elevation Control Mechanism: Setuid and Setgid):** técnica oficial associada a abuso de bit SUID/SGID para escalada de privilégio. <https://attack.mitre.org/techniques/T1548/001/>